

Multi User With OSCGroups Setup

Contents

1	One client per workstation	2
2	The shared-scene rule	2
3	Signal path on every workstation	3
4	Complete workstation pipelines	3
4.1	Workstation A	3
4.2	Workstation B	3
5	Shared and unique settings	3
6	Port map on each workstation	6
7	Before the rehearsal	6
8	Startup procedure on every workstation	6
8.1	1. Start Godot	6
8.2	2. Start one shared local decoder	7
8.2.1	Basic version	7
8.2.2	Advanced allow-list version	7
8.3	3. Configure Bmc for the local performer	7
8.4	4. Start one OscGroupClient	8
8.5	5. Start the local encoder	8
8.6	6. Enable XR-Animator output	8
9	Verify the collaboration	9
10	Avoid feedback and identity errors	9
10.1	Give every encoder a unique identity	9
10.2	Do not send encoder output directly to OSCGroups	9
10.3	Keep Bmc's two outputs together	9

10.4 Keep client receive and transmit ports distinct	9
10.5 Do not relay decoder or client output back to OSCGroups	10
10.6 Use one unique OscGroupClient username per client	10
10.7 Do not expect a server echo	10
11 Diagnose ports	10
12 Shutdown on every workstation	10
13 Related detail	10

This is the canonical setup for collaborative rehearsal and performance between two or more workstations over the Internet. Every workstation runs the same Godot scene and uses the same avatar-to-VMC-port assignments. BuMoChi sends each completed routed frame both to the local decoder and to OSCGroups, so the local and remote Godot instances receive the same motion frames.

1 One client per workstation

One full-duplex `OscGroupClient` handles both network directions:

```
local Bmc -> client localTxPort -> OSCGroups network
OSCGroups network -> client localRxPort -> local decoder -> local Godot
```

Do not start separate sending and receiving clients on an ordinary workstation. Two clients are needed only when deliberately simulating two workstations on one computer, joining two different groups, or using two independent OSCGroups identities.

2 The shared-scene rule

All workstations must use the same Godot scene, avatar identities, and VMC destination ports. For example:

Routed performer	Godot avatar	VMC port
PerformerA	Ishidomaru	39539
PerformerB	Mother	39540

When Workstation A's Bmc completes a **PerformerA** frame, it embeds target port 39539 and sends the exact same routed version-2 message to:

1. its local decoder on 127.0.0.1:39538;
2. its local `OscGroupClient` transmit input on 127.0.0.1:22244.

Workstation B receives the network copy from its `OscGroupClient` directly on decoder port 39538. Both decoders reconstruct the same VMC frame for Godot port 39539. Workstation B applies the same procedure to **PerformerB** on port 39540.

The OSCGroups server normally does not echo a workstation's own message back to it. This is not a problem: Bmc sends its local decoder copy directly.

3 Signal path on every workstation

```
local XR-Animator
-> VMC 127.0.0.1:39537
local BunrakuOSCEncoder
-> route-free version-1 frame 127.0.0.1:57130
local Bmc in SuperCollider
-> routed version-2 frame 127.0.0.1:39538 (local decoder)
-> identical routed version-2 frame 127.0.0.1:22244 (OscGroupClient)
one local OscGroupClient
-> OSCGroups server and remote collaborators

remote routed version-2 frames
-> same local OscGroupClient
-> 127.0.0.1:39538 (local decoder, not Bmc)
one local BunrakuOSCDecoder
-> local Godot VMC ports 39539, 39540, ...
```

The encoder sends only to Bmc. Bmc is the sole authority that adds the final avatar identity and VMC destination before distributing the routed frame locally and over the network.

4 Complete workstation pipelines

The diagrams show a two-workstation example. `PerformerA` and `PerformerB` are stable stream identities; `workstation-a-xr-animator` and `workstation-b-xr-animator` are unique source identities. Both workstations run the same Godot scene: Ishidomaru listens on 39539 and Mother listens on 39540.

4.1 Workstation A

4.2 Workstation B

5 Shared and unique settings

All collaborators must agree on:

- OSCGroups server address and server port, normally 22242;
- group name and password;
- stable performer/avatar and source names;
- the Godot scene;
- the avatar-to-VMC-port map.

Every workstation must have its own:

- OSCGroups username and user password;
- `localToRemotePort` when required by the network/NAT;
- encoder `--avatar` and `--source` identity.

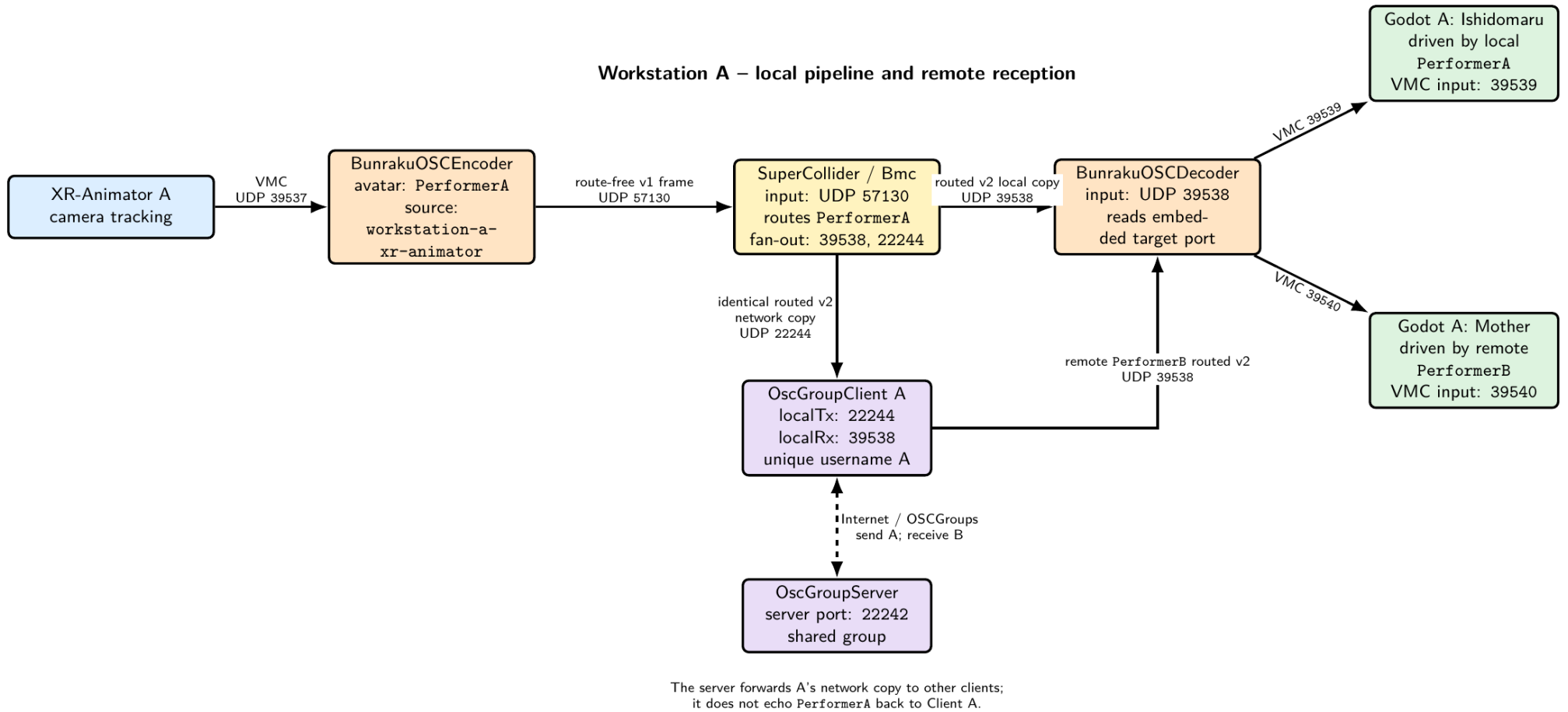


Figure 1: Workstation A routes PerformerA to both local Godot and Workstation B, while receiving PerformerB directly at its decoder.

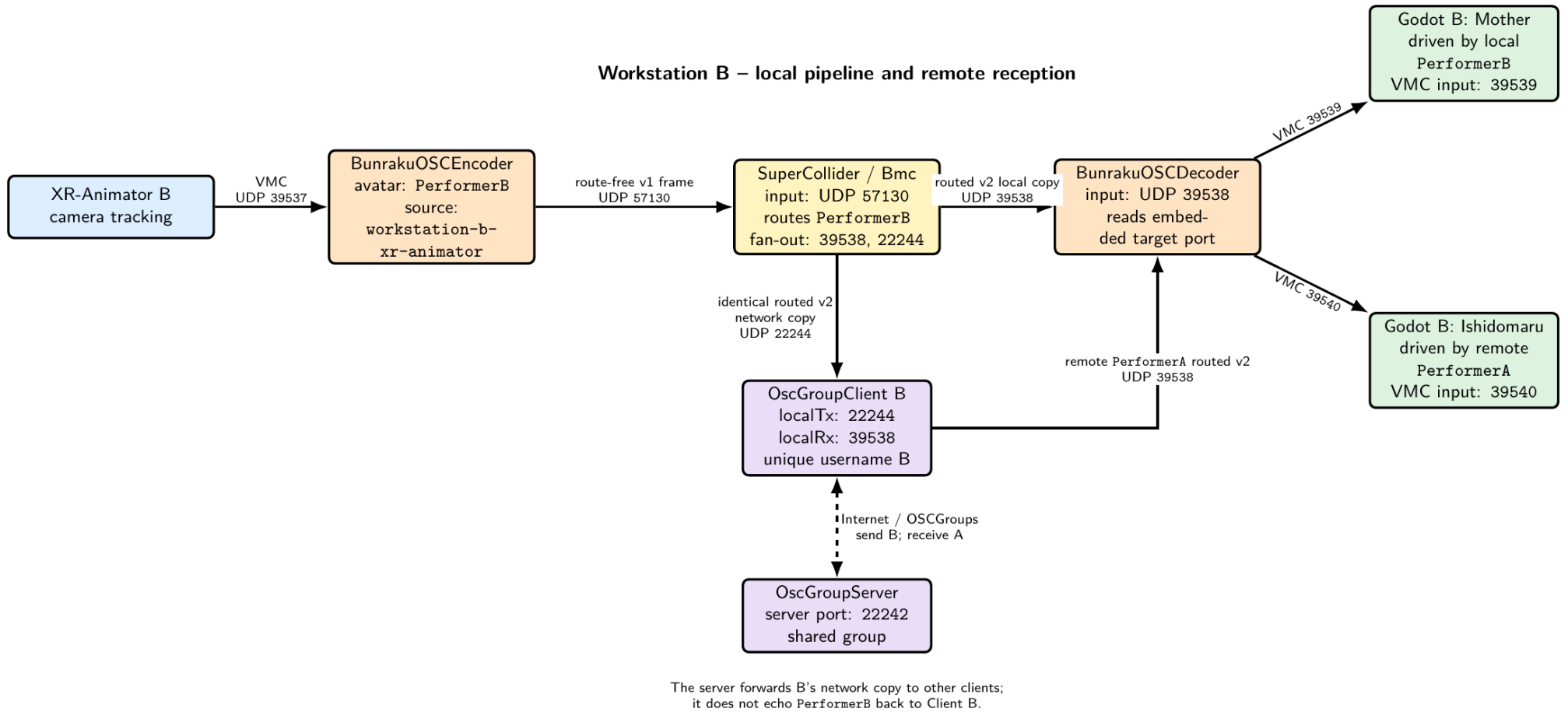


Figure 2: Workstation B routes PerformerB to both local Godot and Workstation A, while receiving PerformerA directly at its decoder.

Local application ports may use the same numbers on different workstations because each computer has its own network stack.

6 Port map on each workstation

Port	Listener	Sender
39537	local <code>BunrakuOSCEncoder</code>	local XR-Animator
57130	local <code>Bmc/SuperCollider</code>	local encoder
22244	local <code>OscGroupClient</code> tx input	local Bmc avatars
39538	local shared decoder	local Bmc and <code>OscGroupClient</code> rx output
39539	first local Godot VMC receiver	local decoder
39540	second local Godot VMC receiver	local decoder

`localToRemotePort` is an additional client port. Choose a free value distinct from 22244 and 39538; collaborators behind the same NAT may need different values.

7 Before the rehearsal

Exchange these values privately:

`SERVER_ADDRESS`

`GROUP_NAME`

`GROUP_PASSWORD`

each workstation's unique `USER_NAME`

each performer's stable `AVATAR_NAME` and `SOURCE_NAME`

the common avatar-to-VMC-port map

Do not store real passwords in this repository. Confirm that every Godot project uses the same scene and port map before testing motion.

8 Startup procedure on every workstation

Perform these steps in order. Commands containing uppercase names are templates.

8.1 1. Start Godot

Open and run the agreed shared scene. Give every independently animated avatar a distinct VMC port and distinct body/face tracker names. In this example:

- Ishidomaru listens on 39539;
- Mother listens on 39540.

8.2 2. Start one shared local decoder

8.2.1 Basic version

```
BunrakuOSCDecoder \  
  --listen-port 39538 \  
  --verbose
```

The decoder accepts the destination embedded in each routed version-2 frame. It receives local frames from `Bmc` and remote frames from `OscGroupClient` on the same UDP port.

8.2.2 Advanced allow-list version

```
BunrakuOSCDecoder \  
  --listen-port 39538 \  
  --allow-target-port 39539 \  
  --allow-target-port 39540 \  
  --verbose
```

Keep the Terminal open.

8.3 3. Configure Bmc for the local performer

Each workstation's `Bmc` processes its own local source. Configure the same target port used for that performer in every copy of the Godot scene. `Bmc` supplies both standard outputs automatically.

Workstation A:

```
(  
Bmc.reset;  
Bmc.addAvatar(\PerformerA, "Ishidomaru");  
Bmc.avatar(\PerformerA).vmcPort_(39539);  
Bmc.start(57130);  
Bmc.status;  
)
```

Workstation B:

```
(  
Bmc.reset;  
Bmc.addAvatar(\PerformerB, "Mother");  
Bmc.avatar(\PerformerB).vmcPort_(39540);  
Bmc.start(57130);  
Bmc.status;  
)
```

Do not replace the avatar's default output function. It fans each routed frame out to the local decoder `127.0.0.1:39538` and `OscGroupClient` `127.0.0.1:22244` destinations.

The fan-out settings are class-wide and may be changed before or during a session:

```
Bmc.decoderPort_(39538);      // default local decoder port
Bmc.oscGroupsPort_(22244);    // default OscGroupClient transmit port
Bmc.forwardDecoder_(true);    // enable/disable the local decoder copy
Bmc.forwardOscGroups_(true);  // enable/disable the network copy
```

The defaults shown above are restored by `Bmc.reset`. Normally no configuration is required. Disable a destination only for deliberate testing or a single-user setup.

8.4 4. Start one OscGroupClient

From the top-level 260715_ICLC27 project directory:

```
AppsAndCode/OSCGroups/bin/macos/OscGroupClient \
  SERVER_ADDRESS \
  22242 \
  LOCAL_TO_REMOTE_PORT \
  22244 \
  39538 \
  USER_NAME \
  USER_PASSWORD \
  GROUP_NAME \
  GROUP_PASSWORD
```

Here 22244 is `localTxPort` and 39538 is `localRxPort`. The client forwards received routed frames directly to the shared decoder. Keep this one client running for both sending and receiving.

8.5 5. Start the local encoder

Workstation A:

```
BunrakuOSCEncoder \
  --avatar "PerformerA" \
  --source "workstation-a-xr-animator" \
  --verbose
```

Workstation B uses `PerformerB` and `workstation-b-xr-animator`. The encoder's only output is local Bmc on 57130; it does not send to OSCGroups.

8.6 6. Enable XR-Animator output

Host: 127.0.0.1

Port: 39537

Enable VMC output.

9 Verify the collaboration

Check from left to right:

1. Each encoder reports increasing `received` and `sent` counts.
2. Each `Bmc.status` reports `running: true` on port 57130.
3. Each `OscGroupClient` reports successful group registration.
4. Each decoder receives local frames from Bmc and remote frames from OSCGroups on 39538.
5. `PerformerA` moves Ishidomaru on 39539 in both Godot scenes.
6. `PerformerB` moves Mother on 39540 in both Godot scenes.

The visual scenes should match apart from ordinary network latency and rendering differences.

10 Avoid feedback and identity errors

10.1 Give every encoder a unique identity

```
# Workstation A
BunrakuOSCEncoder --avatar "PerformerA" --source "workstation-a-xr-animator"
```

```
# Workstation B
BunrakuOSCEncoder --avatar "PerformerB" --source "workstation-b-xr-animator"
```

Do not give both encoders the same `--avatar` and `--source`. Bmc would be unable to distinguish the streams reliably.

10.2 Do not send encoder output directly to OSCGroups

The encoder produces route-free input for Bmc. Do not use the former `--oscgroups-port` route or start a second relay from encoder output to 22244. That would bypass Bmc and distribute frames without the final shared Godot destination.

10.3 Keep Bmc's two outputs together

For every completed avatar frame, Bmc sends an identical routed message to decoder port 39538 and client transmit port 22244. Do not configure different avatar names or target ports for the two destinations.

10.4 Keep client receive and transmit ports distinct

```
localTxPort = 22244    Bmc -> OSCGroups
localRxPort = 39538    OSCGroups -> local decoder
```

Do not set `localRxPort` to 22244: received traffic would re-enter the client's own transmission input and create a network loop. Do not set it to 57130: remote routed frames must not return to Bmc for processing and retransmission.

10.5 Do not relay decoder or client output back to OSCGroups

The decoder's VMC output terminates at Godot. Never send decoder output, or frames received by the client on 39538, back to 22244. Such a return path repeatedly retransmits remote frames.

10.6 Use one unique `OscGroupClient` username per client

Duplicate usernames may replace or confuse a registered endpoint.

10.7 Do not expect a server echo

The group does not need to return a workstation's own frames. Bmc supplies the local decoder copy at the same time that it supplies the network copy.

11 Diagnose ports

```
lsof -nP -iUDP:39537
lsof -nP -iUDP:57130
lsof -nP -iUDP:22244
lsof -nP -iUDP:39538
```

Only one process should listen on each port. Multiple processes may send to decoder port 39538; locally these are Bmc and `OscGroupClient`.

12 Shutdown on every workstation

1. Disable XR-Animator VMC output.
2. Evaluate `Bmc.stop` in SuperCollider.
3. Press `Control-C` in the encoder Terminal.
4. Press `Control-C` in the single `OscGroupClient` Terminal.
5. Press `Control-C` in the decoder Terminal.
6. Stop the running Godot project.

13 Related detail

- [OscGroupClient Use and Configuration](#)
- [OSC Encoder and Decoder Use and Configuration](#)
- [Avatar Port Numbers](#)
- [Port Number Specification](#)
- [Troubleshooting Port Numbers](#)